

PhotoPiler

<https://photopiler.fly.dev/>

Joe Ganley

A few years ago, my daughter had a sleepover with some friends, and one of them had a Polaroid camera. They ended the night with a bunch of photos, and my daughter piled them haphazardly on a desk and took a photo from a low angle and posted that to her Instagram as a memento of the night. Later she said to me: Wouldn't it be cool if you could render an image like that from a set of digital photos? Because of course, people rarely have physical photos any more.

I searched the web and found nothing like this, other than the obvious suggestion of "ask an AI to do it." For now, at least, this is a sufficiently constrained and deterministic problem that applying general AI to it is overkill, both in terms of expense and in terms of bounding the problem space to make it easy for the user. This is entirely within the capabilities of a web app, and that's what I wrote for this project.

As is my custom, I started by writing a fairly detailed functional spec, with an implementation plan that produces a basic MVP as quickly as possible and then iterates to add features. This approach is especially critical in a time-bounded assignment such as this. I fed that spec to Claude in plan mode, and the plan it produced was essentially the one I provided. I suspect I could've just gone straight to implementation from that spec without going through plan mode. I also provided a small piece of code to prototype what I meant by "making a photo look older".

From there we iterated through the phases of the plan, with me inspecting the result and providing feedback after each phase. The bulk of the decisions required were aesthetic choices. Claude does an excellent job of "looking" at the output and judging for itself, but still I made a number of tweaks to make things look better to the human eye.

I spent about 2.5 hours on this, including this writeup and the accompanying video.

If I had more time, the biggest improvement that needs to be made is to make it scale; it loads and processes all of the images in memory, which limits the number and size of images it can handle and also the number of simultaneous users. In a real multiuser app, we would need to store the images in blob storage and process them individually to keep the memory footprint down. With the images stored, we could also precompute and cache the various processed image variations, which would improve the rendering performance as the user worked through various aesthetic settings.